

ЛАБОРАТОРНА РОБОТА № 2

2.1 Тема

Використання операторів умовного виконання та циклів у мові програмування Python.

2.2 Мета роботи

Ознайомитися з алгоритмічними структурами керування в мові Python: операторами умовного виконання **if**, **elif**, **else**, а також циклами **for** і **while**. Набути практичних навичок створення програм з розгалуженням та повторенням дій.

2.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

- знати призначення та синтаксис операторів умовного виконання **if**, **elif**, **else** у мові Python;
- розуміти принципи роботи циклів **for** і **while**, а також особливості їх застосування;
- пояснювати відмінності між циклами з лічильником та циклами з умовою;
- використовувати оператори **break** і **continue** для керування виконанням циклів;
- розробляти прості алгоритми з розгалуженням та повторенням дій;
- реалізовувати програми мовою Python для розв'язання типових задач з використанням умов і циклів;
- аналізувати коректність роботи програми на різних вхідних даних;
- виявляти та виправляти логічні помилки в алгоритмах з умовними операторами та циклами;
- оформлювати програмний код відповідно до базових вимог читабельності.

2.4 Матеріально-технічне забезпечення

- персональний комп'ютер;

- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

2.5 Теоретичні відомості

Оператори умовного виконання в мові Python

У процесі розроблення програм часто виникає потреба виконувати різні дії залежно від певних умов. Для цього в мові програмування Python використовуються оператори умовного виконання, які забезпечують розгалуження алгоритму.

Основними операторами умовного виконання є:

- **if** – оператор перевірки умови;
- **elif** – оператор додаткової умови (скорочення від else if);
- **else** – оператор альтернативної гілки виконання.

Призначення оператора if

Оператор **if** використовується для перевірки логічної умови. Якщо умова є істинною (**True**), виконується відповідний блок коду. У разі, якщо умова хибна (**False**), цей блок коду пропускається.

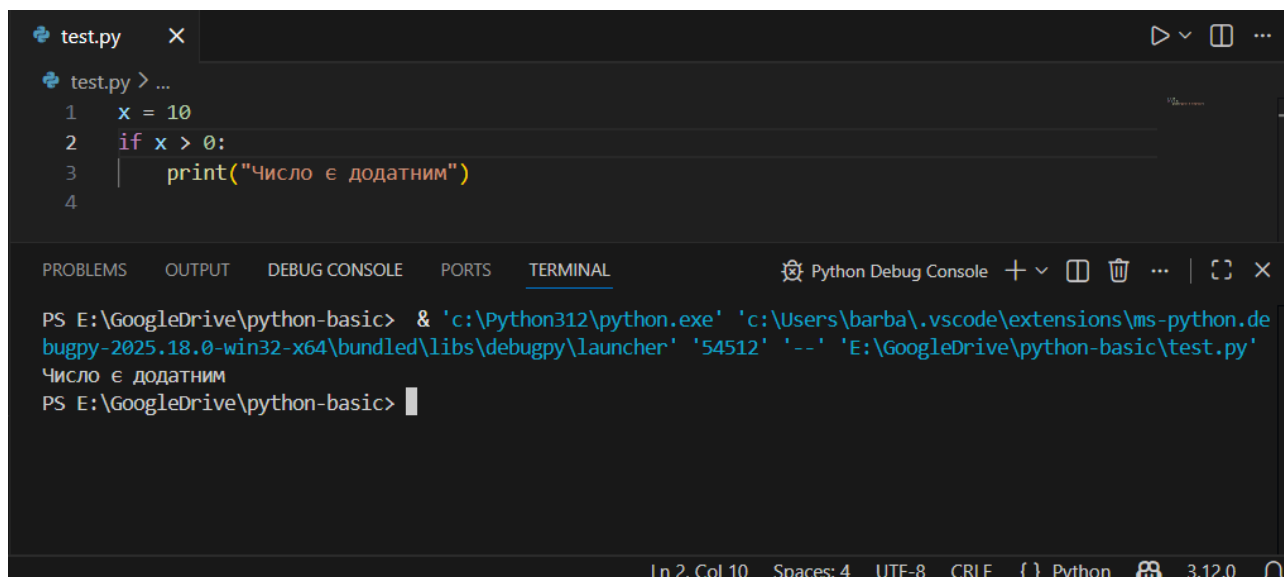
Загальний синтаксис оператора **if**:

```
if умова:  
    блок_інструкцій
```

Умова може містити:

- оператори порівняння (**==**, **!=**, **<**, **>**, **<=**, **>=**);
- логічні оператори (**and**, **or**, **not**);
- вирази, результатом яких є логічне значення.

Приклад використання оператора if:



```
test.py
1 x = 10
2 if x > 0:
3     print("Число є додатним")
4

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
Python Debug Console
PS E:\GoogleDrive\python-basic> & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '54512' '--' 'E:\GoogleDrive\python-basic\test.py'
Число є додатним
PS E:\GoogleDrive\python-basic>

Ln 2, Col 10 Spaces: 4 UTF-8 CRLF {} Python 3.12.0
```

У цьому прикладі повідомлення буде виведене лише тоді, коли значення змінної *x* більше нуля.

Призначення оператора else

Оператор **else** використовується разом з оператором **if** і визначає блок коду, який виконується у випадку, якщо умова в **if** є хибною.

Синтаксис конструкції **if ... else**:

```
if умова:
    блок_інструкцій_1
else:
    блок_інструкцій_2
```

Приклад:

```
number = -5
if number >= 0:
    print("Число невід'ємне")
else:
    print("Число від'ємне")
```

У цьому випадку завжди виконується один з двох блоків коду залежно від результату перевірки умови.

Призначення оператора elif

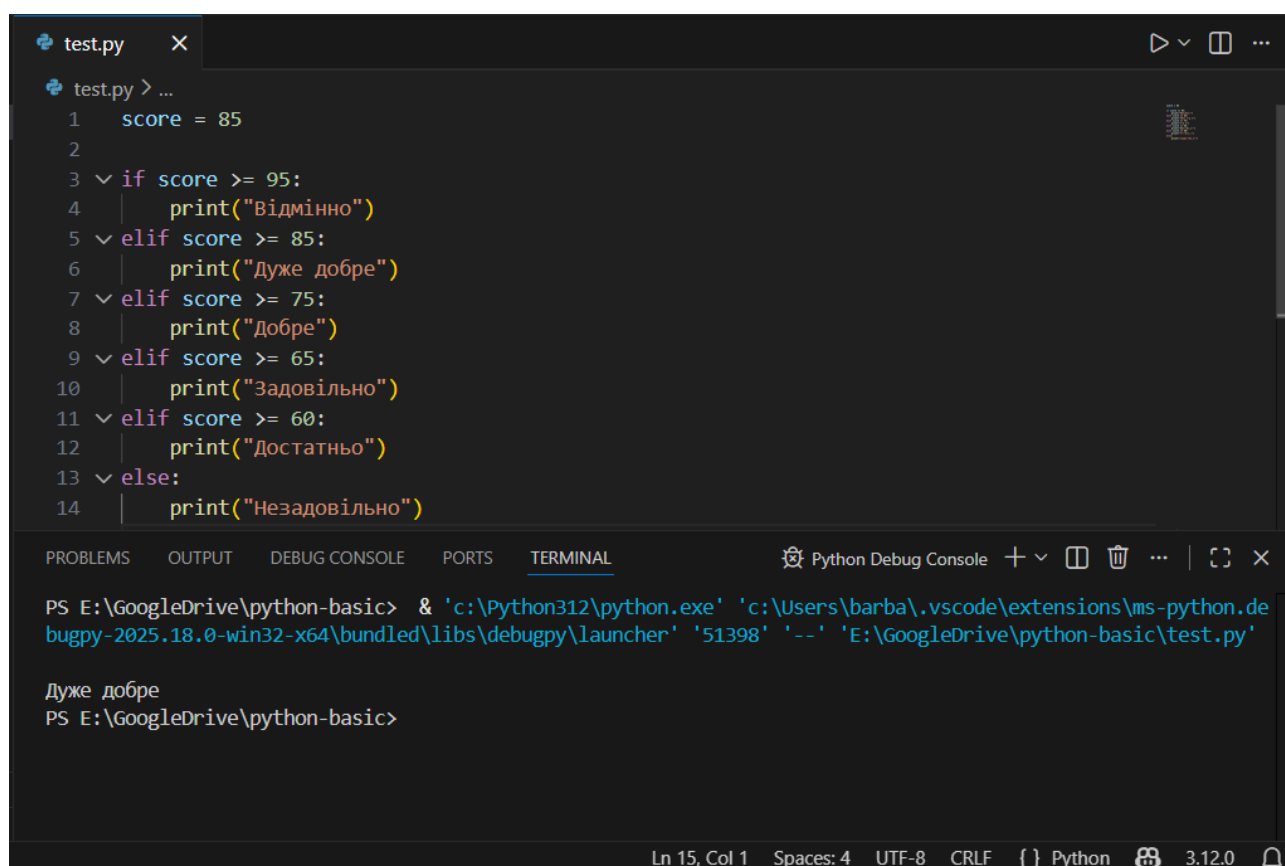
Оператор **elif** застосовується у випадках, коли необхідно перевірити декілька альтернативних умов. Він дозволяє уникнути вкладених конструкцій **if** і підвищує читабельність програмного коду.

Загальний синтаксис конструкції *if ... elif ... else*:

```
if умова_1:
    блок_інструкцій_1
elif умова_2:
    блок_інструкцій_2
elif умова_3:
    блок_інструкцій_3
else:
    блок_інструкцій_4
```

Перевірка умов відбувається послідовно зверху вниз. Виконується лише той блок коду, умова якого першою набуває значення **True**. Якщо жодна з умов не є істинною, виконується блок **else** (якщо він присутній).

Приклад:



The screenshot shows a VS Code editor window with a file named `test.py`. The code in the editor is as follows:

```
1 score = 85
2
3 if score >= 95:
4     print("Відмінно")
5 elif score >= 85:
6     print("Дуже добре")
7 elif score >= 75:
8     print("Добре")
9 elif score >= 65:
10    print("Задовільно")
11 elif score >= 60:
12    print("Достатньо")
13 else:
14    print("Незадовільно")
```

Below the editor, the **TERMINAL** panel shows the command used to run the script and its output:

```
PS E:\GoogleDrive\python-basic> & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51398' '--' 'E:\GoogleDrive\python-basic\test.py'

Дуже добре
PS E:\GoogleDrive\python-basic>
```

The status bar at the bottom indicates the file is at line 15, column 1, with 4 spaces, UTF-8 encoding, CRLF line endings, and the Python 3.12.0 interpreter.

Особливості синтаксису умовних операторів у Python

1) Відступи (indentation). Python використовує відступи для визначення блоків коду. Усі інструкції, що належать до умовного оператора, повинні мати однаковий відступ (зазвичай 4 пробіли).

2) Відсутність фігурних дужок. На відміну від багатьох інших мов програмування, у Python не використовуються фігурні дужки `{ }` для обмеження блоків коду.

3) Обов'язковий символ двокрапки (:). Після умови в операторах ***if***, ***elif***, ***else*** завжди ставиться двокрапка.

4) Умова як логічний вираз. Умовою може бути будь-який вираз, що повертає логічне значення, або об'єкт, який може бути інтерпретований як ***True*** або ***False***.

Вкладені умовні оператори

За потреби умовні оператори можуть вкладатися один в один, що дозволяє реалізовувати складніші алгоритми.

Приклад вкладеного ***if***:

```
x = 5

if x > 0:
    if x % 2 == 0:
        print("Число додатне і парне")
    else:
        print("Число додатне і непарне")
```

Проте надмірне вкладення умов знижує читабельність коду, тому у більшості випадків доцільно використовувати оператор ***elif***.

Оператори умовного виконання ***if***, ***elif***, ***else*** є базовими інструментами керування логікою виконання програм у мові Python. Їх правильне застосування дозволяє створювати алгоритми, які реагують на різні вхідні дані, забезпечуючи гнучкість та коректність роботи програм.

Циклічні алгоритми в програмуванні

У програмуванні часто виникає потреба багаторазово виконувати один і той самий набір інструкцій. Такі задачі розв'язуються за допомогою циклів – алгоритмічних конструкцій, що забезпечують повторення дій за певною умовою або визначену кількість разів.

У мові програмування Python основними операторами циклів є:

- цикл ***for***;
- цикл ***while***.

Кожен з них має свої особливості синтаксису та області доцільного застосування.

Цикл **for** у мові Python

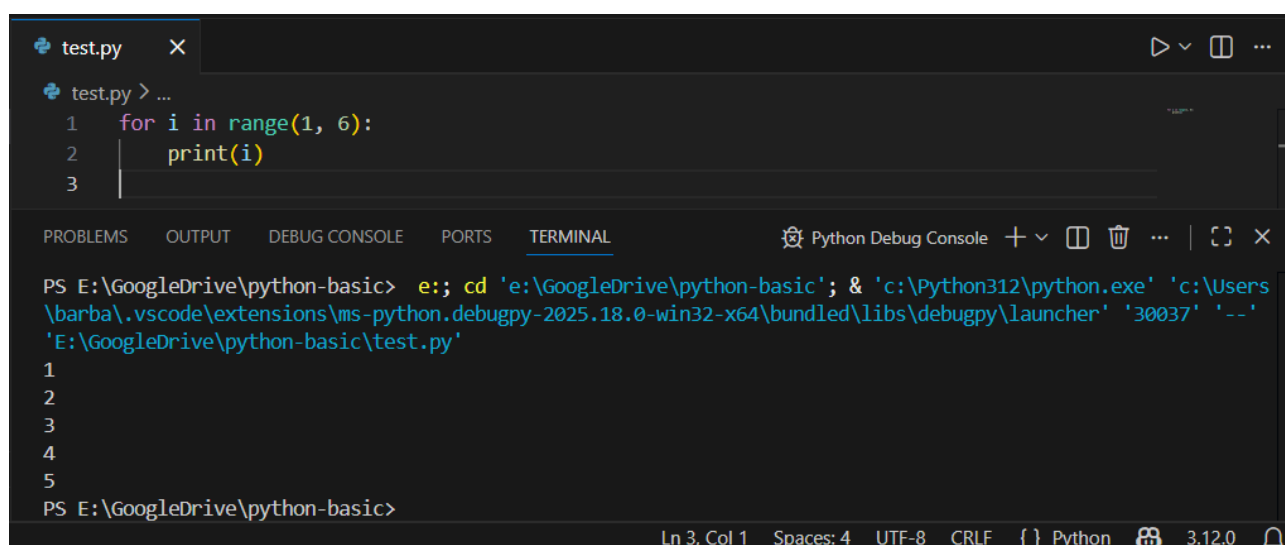
Цикл **for** використовується для послідовного перебору елементів ітерабельних об'єктів, таких як діапазони чисел, списки, кортежі, рядки, словники.

Загальний синтаксис циклу **for**:

```
for змінна in послідовність:
    блок_інструкцій
```

На кожній ітерації циклу змінна циклу послідовно набуває значення чергового елемента з послідовності, після чого виконується тіло циклу.

Найпоширенішим випадком використання циклу **for** є перебір числового діапазону за допомогою функції **range()**:



The screenshot shows a VS Code editor with a file named `test.py`. The code in the editor is:

```
1 for i in range(1, 6):
2     print(i)
3
```

Below the editor, the `TERMINAL` panel shows the command used to run the script:

```
PS E:\GoogleDrive\python-basic> e;; cd 'e:\GoogleDrive\python-basic'; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '30037' '--' 'E:\GoogleDrive\python-basic\test.py'
```

The output of the script is shown as numbers 1 through 5, each on a new line.

У даному прикладі цикл виконується фіксовану кількість разів, що наперед відома.

Особливості циклу **for**:

- кількість повторень, як правило, відома до початку виконання циклу;
- зручний для обробки послідовностей даних;
- забезпечує читабельність та компактність коду;
- автоматично переходить до наступного елемента без додаткових умов.

Цикл **while** у мові Python

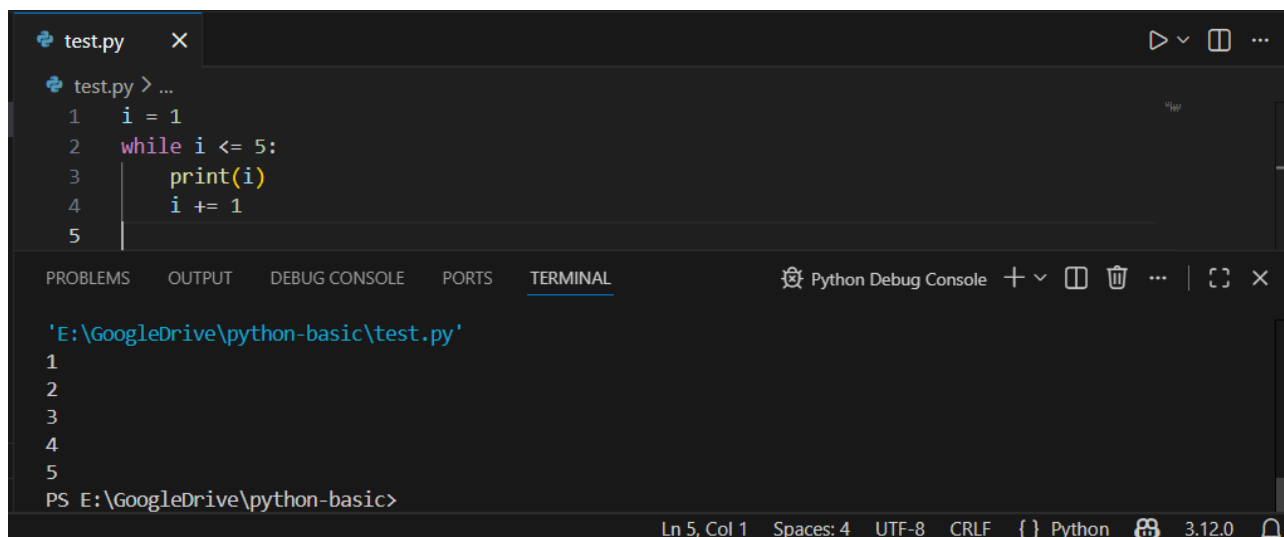
Цикл **while** використовується для повторення дій доти, доки виконується певна логічна умова.

Загальний синтаксис циклу **while**:

```
while умова:
    блок_інструкцій
```

Перед кожною ітерацією перевіряється умова циклу. Якщо вона істинна, виконується тіло циклу; якщо хибна – виконання циклу припиняється.

Приклад використання циклу **while**:



```
test.py
test.py > ...
1 i = 1
2 while i <= 5:
3     print(i)
4     i += 1
5

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL Python Debug Console
'E:\GoogleDrive\python-basic\test.py'
1
2
3
4
5
PS E:\GoogleDrive\python-basic>
Ln 5, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.12.0
```

У цьому випадку кількість ітерацій не задана явно і залежить від виконання умови.

Особливості циклу **while**:

- кількість повторень заздалегідь може бути невідомою;
- використовується для циклів з умовою завершення;
- потребує уважного контролю зміни змінних умови;
- існує ризик створення нескінченного циклу.

Порівняння циклів **for** і **while**

Критерій	Цикл for	Цикл while
Кількість повторень	Відома наперед	Невідома наперед
Основне призначення	Перебір послідовностей	Повторення за умовою
Простота використання	Вища	Нижча
Ризик нескінченного циклу	Мінімальний	Підвищений

Таким чином, цикл **for** доцільно застосовувати тоді, коли відома кількість ітерацій або необхідно перебрати елементи колекції, а цикл **while** – коли повторення залежать від динамічної умови.

Цикли **for** і **while** є фундаментальними елементами алгоритмічної структури програм у Python. Розуміння принципів їх роботи дозволяє будувати ефективні алгоритми для обробки даних, керування процесами та реалізації складних логічних рішень. Правильний вибір типу циклу під конкретну задачу є важливою умовою створення коректних і зрозумілих програм.

Оператори керування виконанням циклів **break** і **continue** у Python

Під час програмування часто виникає необхідність достроково змінювати хід виконання циклу залежно від певних умов. Для цього в мові програмування Python використовуються спеціальні оператори керування циклом – **break** та **continue**. Вони дозволяють зробити алгоритм більш гнучким, ефективним і зрозумілим.

Оператор **break** призначений для негайного припинення виконання циклу, в якому він знаходиться. Після виконання **break** програма виходить з тіла циклу та переходить до виконання першої інструкції, що знаходиться після циклу.

Оператор **break** може використовуватися як у циклі **for**, так і в циклі **while**. Найчастіше він застосовується у випадках, коли:

- необхідне значення вже знайдено;
- подальше виконання циклу не має сенсу;
- виникла помилка або некоректна ситуація, яка потребує припинення повторень.

Загальний принцип роботи оператора **break**:

- 1) виконується перевірка умови всередині циклу;
- 2) у разі виконання умови виконується оператор **break**;
- 3) цикл завершується незалежно від кількості запланованих ітерацій.

Приклад логіки застосування **break**:

- пошук числа у послідовності – після знаходження потрібного значення немає необхідності продовжувати перегляд інших елементів.

Оператор **continue** використовується для пропуску поточної ітерації циклу та негайного переходу до наступної ітерації. При цьому сам цикл не припиняється, а лише пропускається виконання коду, що знаходиться після **continue** в тілі циклу.

Оператор ***continue*** застосовується у випадках, коли:

- певні значення не потребують обробки;
- необхідно пропустити елементи, що не відповідають заданій умові;
- логіка обробки даних передбачає фільтрацію значень.

Загальний принцип роботи оператора ***continue***:

- 1) перевіряється умова всередині циклу;
- 2) у разі виконання умови виконується оператор ***continue***;
- 3) керування передається на початок наступної ітерації циклу.

Приклад логіки застосування ***continue***:

- виведення чисел у діапазоні з пропуском певних значень (наприклад, кратних 3).

Порівняння операторів ***break*** і ***continue***

Характеристика	<i>break</i>	<i>continue</i>
Вплив на цикл	Повністю завершує цикл	Пропускає одну ітерацію
Подальше виконання	Переходить за межі циклу	Переходить до наступної ітерації
Типове застосування	Пошук, аварійне завершення	Фільтрація, пропуск даних

Особливості використання в циклах ***for*** і ***while***:

- У циклі ***for*** оператори ***break*** і ***continue*** застосовуються під час перебору елементів послідовності.
- У циклі ***while*** слід бути особливо уважним при використанні ***continue***, оскільки пропуск коду може призвести до нескінченного циклу, якщо змінна умови не оновлюється.

Методичні зауваження

- Оператори ***break*** і ***continue*** слід застосовувати усвідомлено, не порушуючи логіку алгоритму.
- Надмірне використання цих операторів може ускладнювати читання програми.
- Рекомендується поєднувати ***break*** і ***continue*** з чіткими умовами ***if***, що підвищує зрозумілість коду.

Оператори ***break*** і ***continue*** є важливими інструментами керування виконанням циклів у мові Python. Вони дозволяють оптимізувати алгоритми, уникати зайвих обчислень і реалізовувати складні логічні умови обробки даних. Володіння цими операторами є необхідною складовою формування навичок програмування з використанням циклічних конструкцій.

Оформлення програмного коду та вимоги до його читабельності

Читабельність програмного коду є однією з ключових вимог до якісного програмного продукту. Навіть правильно працююча програма може вважатися неякісною, якщо її код складно зрозуміти, модифікувати або перевірити. У мові програмування Python особлива увага приділяється зрозумілості та акуратності коду, оскільки він часто використовується для командної розробки та навчання.

Поняття читабельності програмного коду

Читабельність коду – це властивість програмного тексту бути зрозумілим для іншого програміста або для самого автора через певний час. Читабельний код:

- легко сприймається візуально;
- має логічну структуру;
- використовує зрозумілі імена змінних та функцій;
- не містить зайвих або неочевидних конструкцій.

Python орієнтований на написання саме читабельного коду, що відображено у філософії мови («Zen of Python»), де одним із ключових принципів є твердження: «Читабельність має значення».

Відступи та блокова структура програми

У Python відступи є не лише елементом оформлення, а й частиною синтаксису мови. Вони визначають межі логічних блоків коду.

Основні правила:

- для відступів використовується 4 пробіли;
- заборонено змішувати пробіли та табуляцію;
- кожен вкладений блок (умова, цикл, функція) має бути зсунений праворуч відносно попереднього рівня.

Основи розробки програмного забезпечення мовою програмування Python.

к.т.н., доц. Барбарук Віктор Миколайович

Приклад правильно оформленого блоку:

```
if number > 0:
    print("Число додатне")
else:
    print("Число від'ємне або нуль")
```

Неправильні або відсутні відступи призводять до синтаксичних помилок або ускладнюють розуміння логіки програми.

Іменування змінних та функцій

Зрозумілі імена змінних значно підвищують читабельність коду. У Python прийнято використовувати змістовні імена, які відображають призначення змінної або функції.

Рекомендації:

- імена змінних пишуться малими літерами;
- для складених назв використовується символ підкреслення (snake_case);
- імена повинні відображати сутність даних.

Приклади:

- коректно: total_sum, student_age, is_prime
- некоректно: a, x1, var, tmp

Функції також повинні мати зрозумілі назви, що описують дію, яку вони виконують.

Коментарі у програмному коді

Коментарі використовуються для пояснення логіки роботи програми або окремих її фрагментів. Вони не впливають на виконання коду, але значно полегшують його аналіз.

У Python використовуються:

- однорядкові коментарі, що починаються з символу #;
- багаторядкові рядки документації (docstring), які зазвичай застосовуються для функцій.

Коментарі повинні:

- пояснювати складні або неочевидні ділянки коду;

- бути короткими та зрозумілими;
- не дублювати очевидні дії.

Надмірна кількість коментарів або їх відсутність негативно впливає на читабельність.

Форматування операторів та виразів

Для покращення сприйняття коду необхідно дотримуватися правил форматування:

- між операторами та операндами ставляться пробіли;
- довгі вирази бажано розбивати на декілька рядків;
- один рядок – одна логічна дія.

Приклад:

```
average = (value1 + value2 + value3) / count
```

Таке форматування робить програму більш зрозумілою та акуратною.

Логічна структура програми

Програма повинна мати чітку логічну структуру:

- 1) введення даних;
- 2) обробка даних;
- 3) виведення результатів.

Кожен логічний етап бажано відокремлювати порожнім рядком. Це полегшує орієнтацію в коді та дозволяє швидше зрозуміти його алгоритм.

Дотримання стандартів оформлення коду

У Python існують загальноприйняті рекомендації з оформлення коду, зокрема стандарт PEP 8, який визначає базові правила стилю. На початковому етапі вивчення мови достатньо дотримуватись основних положень:

- відступи – 4 пробіли;
- довжина рядка – не більше 79 символів;
- зрозумілі імена змінних;

— акуратне використання коментарів.

Дотримання стандартів оформлення сприяє формуванню професійних навичок програмування та підвищує якість програмного коду.

Оформлення програмного коду відповідно до базових вимог читабельності є обов'язковою складовою процесу програмування. Дотримання правил відступів, іменування, коментування та структуризації коду дозволяє створювати зрозумілі, логічні та легко супроводжувані програми мовою Python.

2.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий програмний файл.
- 3) Реалізувати введення початкових даних з клавіатури.
- 4) Використати оператори умовного виконання та/або цикли відповідно до завдання.
- 5) Забезпечити коректне виведення результатів на екран.
- 6) Перевірити роботу програми на різних вхідних даних.
- 7) Зберегти файл з програмним кодом.
- 8) Оформити звіт до лабораторної роботи.

2.7 Загальне завдання

Написати програму, яка:

- вводить початкові дані;
- виконує розрахунки згідно індивідуального завдання;
- виводить результату

2.8 Індивідуальні завдання

- 1) Для заданого першого члена та різниці прогресії вивести значення заданої кількості членів арифметичної прогресії.
- 2) Обчислити факторіал заданого числа за допомогою циклу.

- 3) Знайти прості числа в заданому інтервалі.
- 4) Вивести послідовність чисел Фібоначчі до заданого значення.
- 5) Визначити, чи є введене число додатним або від'ємним.
- 6) Знайти найбільше число серед трьох введених.
- 7) Вивести числа від 1 до 100, пропускаючи кратні 3.
- 8) Побудувати таблицю множення для заданого числа.
- 9) Перевірити рік на високосність.
- 10) 1) Для заданого першого члена та знаменника прогресії вивести значення заданої кількості членів геометричної прогресії.

Нульовий варіант (зразок виконання)

Завдання:

Написати програму мовою Python, яка вводиться з клавіатури ціле число та визначає, чи є воно додатним, від'ємним або нульовим з використанням операторів умовного виконання if, elif, else.

Алгоритм виконання:

- 1) Ввести ціле число з клавіатури.
- 2) Перевірити значення введеного числа:
 - якщо число більше нуля — вивести повідомлення «Число додатне»;
 - якщо число менше нуля — вивести повідомлення «Число від'ємне»;
 - якщо число дорівнює нулю — вивести повідомлення «Число дорівнює нулю».
- 3) Завершити роботу програми.

Програмний код (приклад реалізації)

```
# Введення цілого числа з клавіатури
number = int(input("Введіть ціле число: "))

# Перевірка, чи є число додатним
if number > 0:
    print("Число є додатним")

# Перевірка, чи є число від'ємним
```

```
elif number < 0:
    print("Число є від'ємним")

# Випадок, коли число дорівнює нулю
else:
    print("Число дорівнює нулю")
```

Результат роботи

```
PS E:\GoogleDrive\python-basic> & C:/Python312/python.exe e:/GoogleDr
ive/python-basic/test.py
Введіть ціле число: -7
Число є від'ємним
PS E:\GoogleDrive\python-basic> |
```

Пояснення до коду

- функція `input()` використовується для введення даних з клавіатури;
- Функція `int()` перетворює введений рядок у ціле число;
- оператор `if` перевіряє основну умову (`number > 0`);
- оператор `elif` використовується для альтернативної умови (`number < 0`);
- оператор `else` виконується у випадку, якщо жодна з попередніх умов не виконалась;
- відступи (4 пробіли) визначають блоки коду, що належать до кожної умови.

2.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву дисципліни та номер лабораторної роботи.
- 2) Тему та мету роботи.
- 3) Умову індивідуального завдання.
- 4) Алгоритм розв'язання (словесно або у вигляді блок-схеми).
- 5) Лістинг програми мовою Python.

6) Приклади вхідних та вихідних даних.

7) Висновок щодо виконаної роботи.

2.10 Контрольні питання

- 1) Як працює оператор умовного виконання if у Python?
- 2) У чому полягає різниця між циклами for та while?
- 3) Яке призначення оператора break?
- 4) Для чого використовується оператор continue?
- 5) Як реалізувати алгоритм перевірки числа на простоту?

2.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>
- 2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>